Listing 5.3 Create knowledge graph shown in figure 5.7 as a labeled property graph and run a traversal over it.

```
import mgclient

conn = mgclient.connect(host='127.0.0.1', port=7687) #1
c = conn.cursor()

c.execute("MATCH (n) DETACH DELETE n") #2
c.execute("""
CREATE (hotel:Building {name: 'Hotel'})
CREATE (l1:Level {name: 'Level 1'})
CREATE (l2:Level {name: 'Level 2'})
CREATE (room:Room {name: 'Room 209'})
CREATE (thermo:Equipment {name: 'Thermostat 24967', WiFiSignalStrengt
h: -70,
➥ BatteryPercentage: 83})
CREATE (l1)-[:isPartOf]->(hotel)
CREATE (l2)-[:isPartOf]->(hotel)
CREATE (room)-[:isPartOf]->(l2)
CREATE (thermo)-[:locatedIn]->(room)
""")

c.execute( #3
  """
   MATCH (l:Level {name: 'level2'})<-[:isPartOf]-(room)-[:locatedIn]-
(
   ➥e:Equipment)
   RETURN l.name, e.name, e.BatteryPercentage
  """
)

for row in c.fetchall():
    print(f"{row[0]} has {row[1]} (Battery: {row[2]}%)")
conn.close()
```

#1 Connect to the graph database running locally in Docker.
#2 Clear and create the knowledge graph.
#3 Traverse the graph to get the battery level of all Equipment on level two of the hotel.

The traversal shown in listing 5.3 can be thought of in plain language as *get me all equipment in rooms that are located on a level named level2 in the hotel*, with the full anatomy of the Cypher statement shown below.